

ELASTIC STACK SIEM Deployment Guide

Elasticsearch, Kibana & Logstash — Single-Node Installation on Ubuntu Server

Version 9.4.2 | Step-by-Step Hands-On Lab | June 2026
Deployed on Ubuntu Server (VM) — Host: elasticsearch

Prepared by

Khizar UI Islam (Malik)

BS Software Engineering — SZABIST Islamabad | CEH Certified
GitHub: Khizar-malik97 | mrkhizar97@gmail.com

Table of Contents

- 1. Introduction & Lab Overview**
- 2. Architecture: How Elasticsearch, Kibana & Logstash Fit Together**
- 3. Part 1 — Preparing the Host (System Update)**
- 4. Part 2 — Adding the Elastic Package Repository**
- 5. Part 3 — Installing Elasticsearch**
- 6. Part 4 — Installing Kibana & Logstash**
- 7. Part 5 — Configuring Elasticsearch (elasticsearch.yml)**
- 8. Part 6 — Configuring Kibana (kibana.yml)**
- 9. Part 7 — Starting and Verifying the Services**
- 10. Part 8 — Enrolling Kibana with Elasticsearch**
- 11. Part 9 — First Login and Stack Verification**
- 12. Conclusion & Recommended Next Steps**

1. Introduction & Lab Overview

Centralized logging and search sit at the core of almost every modern SIEM and security monitoring stack. The Elastic Stack — Elasticsearch, Kibana, and Logstash, commonly referred to as the “ELK Stack” — is one of the most widely deployed open-source platforms for ingesting, indexing, searching, and visualizing large volumes of log and event data in real time.

This guide documents a complete, hands-on deployment of the Elastic Stack on a single Ubuntu Server host. It walks through every stage exactly as it was performed: updating the operating system, adding Elastic's official package repository, installing Elasticsearch, Kibana, and Logstash, configuring each component for network access, starting and verifying the services with systemd, and finally enrolling Kibana with Elasticsearch through the browser-based setup wizard. Every command that was run, and every screen that resulted from it, is captured and explained below.

Lab Environment

Operating System	Ubuntu Server (APT-based)
Hostname	elasticsearch
Primary User	malik
Elasticsearch Version	9.4.2
Companion Services	Kibana 9.4.2, Logstash
Kibana Web Port	5601
Elasticsearch API Port	9200
Host IP (lab)	192.168.44.134

The deployment follows Elastic's official Debian/APT package installation method rather than a Docker container or the all-in-one assisted installer. This mirrors how the Elastic Stack is most commonly installed in real production and SOC environments running on Linux servers, and it gives full visibility into every configuration file that gets touched along the way.

2. Architecture: How the Components Fit Together

Before installing anything, it helps to understand what each component is responsible for and how data flows between them in a typical SIEM pipeline:

- Elasticsearch — the search and analytics engine. It stores all indexed data (logs, events, alerts) and serves the queries that power dashboards and searches. This is the heart of the stack.

- Kibana — the web-based visualization and management interface. It connects to Elasticsearch and lets analysts search data, build dashboards, and manage the cluster through a browser, on port 5601 by default.
- Logstash — the data processing pipeline. It can ingest logs and events from many sources, parse and transform them (filtering, enriching, normalizing formats), and forward the structured result into Elasticsearch for indexing.

In this single-node lab, all three components are installed on the same Ubuntu Server VM. Elasticsearch and Kibana are both reconfigured to listen on the host's network interface (0.0.0.0) rather than only on localhost, which allows the Kibana web UI to be reached from a browser on another machine on the same network — the same pattern used when accessing a SIEM dashboard remotely in a real SOC.

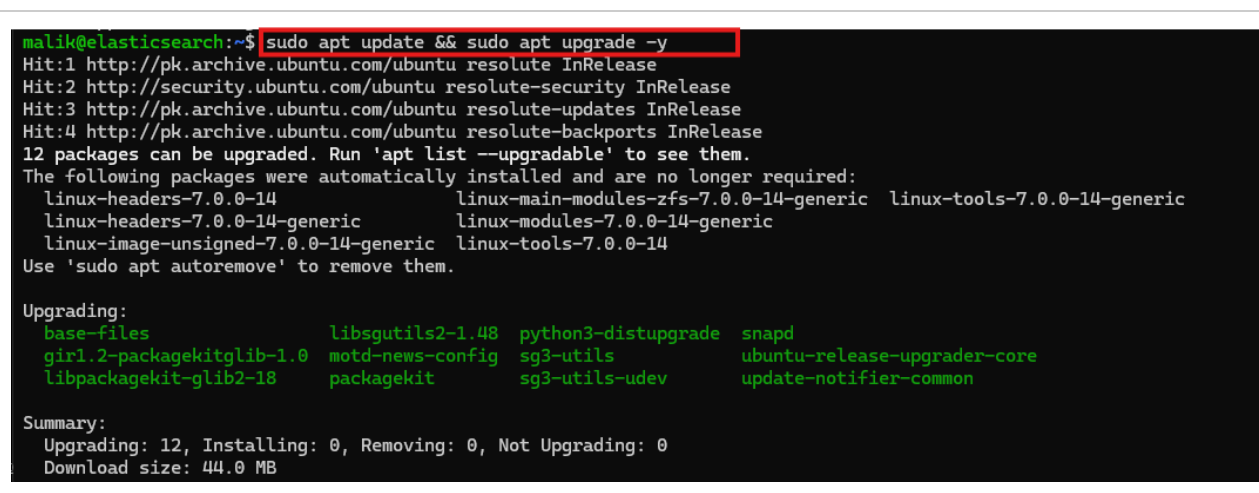
3. Part 1 — Preparing the Host

Every clean installation starts with making sure the operating system itself is fully patched. Running the package manager update before adding any third-party repository avoids dependency conflicts later in the install.

Command — *Update the OS package index and upgrade installed packages*

```
sudo apt update && sudo apt upgrade -y
```

Purpose: Refreshes the list of available packages and installs all pending updates.



```
malik@elasticsearch:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://pk.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:3 http://pk.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://pk.archive.ubuntu.com/ubuntu resolute-backports InRelease
12 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14          linux-main-modules-zfs-7.0.0-14-generic  linux-tools-7.0.0-14-generic
  linux-headers-7.0.0-14-generic  linux-modules-7.0.0-14-generic
  linux-image-unsigned-7.0.0-14-generic  linux-tools-7.0.0-14
Use 'sudo apt autoremove' to remove them.

Upgrading:
  base-files          libsgutils2-1.48  python3-distupgrade  snapd
  gir1.2-packagekitglib-1.0  motd-news-config  sg3-utils            ubuntu-release-upgrader-core
  libpackagekit-glib2-18    packagekit        sg3-utils-udev       update-notifier-common

Summary:
Upgrading: 12, Installing: 0, Removing: 0, Not Upgrading: 0
Download size: 44.0 MB
```

Figure 1 – Terminal output of 'sudo apt update && sudo apt upgrade -y', showing 12 packages queued for upgrade and a 44.0 MB download size.

Locating the Official Installation Documentation

With the base system updated, the next step was confirming the correct installation method from Elastic's own documentation rather than relying on third-party tutorials. Searching for the official guide surfaced Elastic's "Install Elasticsearch with a Debian package" page, which documents the supported APT-based installation path for Debian and Ubuntu systems.

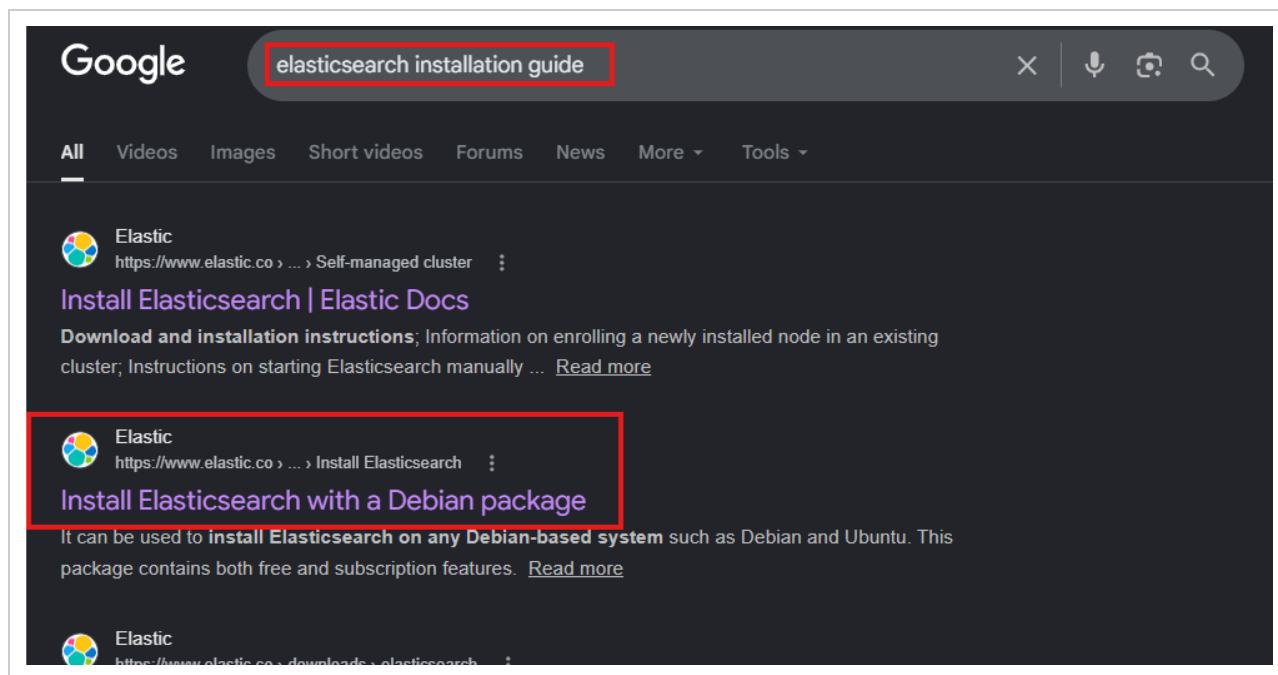


Figure 2 – Search results pointing to Elastic's official documentation for installing Elasticsearch via the Debian/APT package.

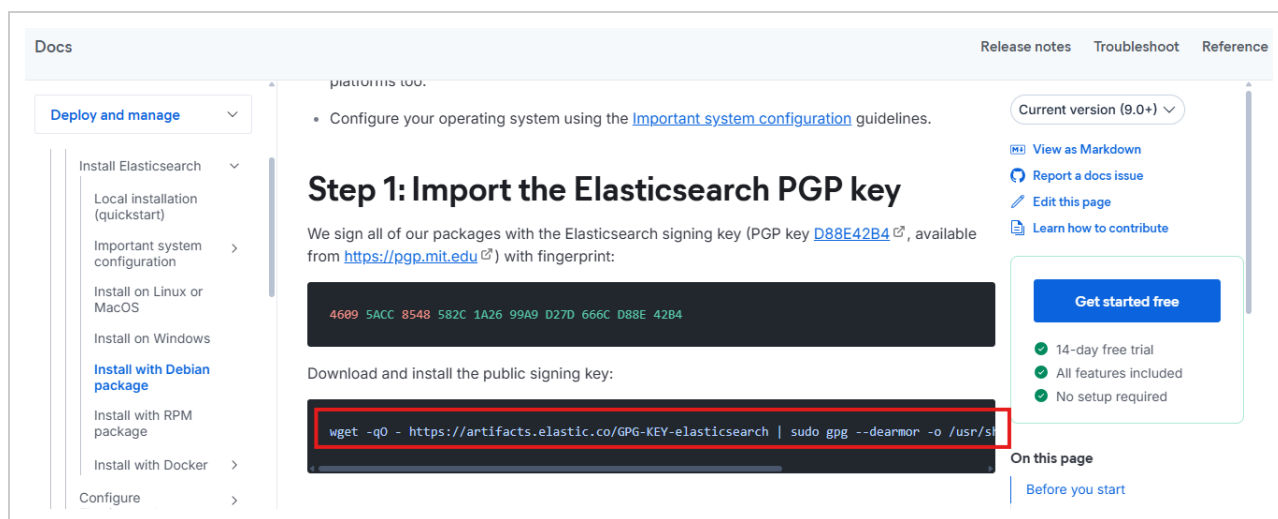


Figure 3 – Elastic's official documentation, Step 1: importing the Elasticsearch PGP signing key before adding the repository.

4. Part 2 — Adding the Elastic Package Repository

Elastic distributes its packages through its own APT repository rather than Ubuntu's default repositories. Before apt can trust and download from that repository, its GPG signing key has to be imported, the repository definition has to be added to the system's sources list, and the package index has to be refreshed.

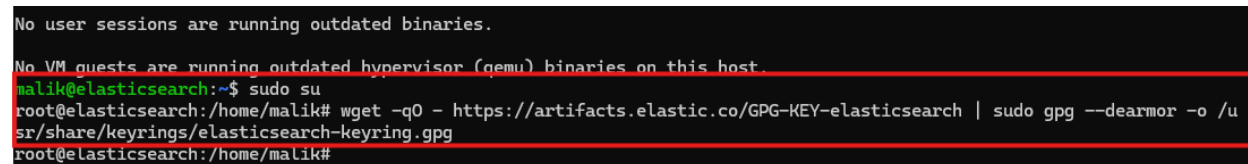
Step 4.1 – Switching to Root and Importing the Elastic GPG Key

The terminal was switched to the root user with `sudo su` so that the remaining setup commands didn't each need a `sudo` prefix. The Elastic signing key was then downloaded with `wget` and piped directly into `gpg`, which dearmors it and writes it into the system keyring at `/usr/share/keyrings/elasticsearch-keyring.gpg`.

Command

```
sudo su
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch \
| sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
```

Purpose: Downloads Elastic's signing key and adds it to the system so apt can trust Elastic's packages.

A terminal window screenshot with a black background and white text. At the top, two system messages are displayed: "No user sessions are running outdated binaries." and "No VM guests are running outdated hypervisor (qemu) binaries on this host." Below these, the user 'malik' at host 'elasticsearch' enters the command 'sudo su'. The prompt changes to 'root@elasticsearch:~/home/malik#'. The user then enters a long command: 'wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg'. The prompt changes to 'root@elasticsearch:~/home/malik#'. A red rectangular box highlights the command and its execution. The terminal output is as follows:

```
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
malik@elasticsearch:~$ sudo su
root@elasticsearch:~/home/malik# wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /u
sr/share/keyrings/elasticsearch-keyring.gpg
root@elasticsearch:~/home/malik#
```

Figure 4 – Terminal showing the switch to the root user, followed by the `wget | gpg --dearmor` command importing the Elasticsearch signing key into the local keyring.

Step 4.2 – Installing apt-transport-https and Defining the Repository

Elastic's repository is served over HTTPS, so the `apt-transport-https` package is required so apt can fetch package lists and `.deb` files over a TLS-encrypted connection. The documentation's next step is to save the actual repository definition into its own file under `/etc/apt/sources.list.d/`.

Install from the APT repository

1. You may need to install the `apt-transport-https` package on Debian before proceeding:

```
sudo apt-get install apt-transport-https
```

2. Save the repository definition to `/etc/apt/sources.list.d/elastic-9.x.list`:

```
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/9.x/apt stable main" | \
```

Figure 5 – Elastic documentation showing the two remaining setup steps: installing `apt-transport-https`, and saving the repository definition to `/etc/apt/sources.list.d/elastic-9.x.list`.

Command — Install HTTPS transport support for APT

```
sudo apt-get install apt-transport-https
```

Purpose: Installs HTTPS support so apt can download packages from Elastic's repository securely.

```
root@elasticsearch:/home/malik# sudo apt-get install apt-transport-https
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14 linux-headers-7.0.0-14-generic linux-image-unsigned-7.0.0-14-generic
  linux-main-modules-zfs-7.0.0-14-generic linux-modules-7.0.0-14-generic linux-tools-7.0.0-14
  linux-tools-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  apt-transport-https
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 3,928 B of archives.
After this operation, 36.9 kB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu/resolute/universe amd64 apt-transport-https all 3.2.0 [3,928 B]
Fetched 3,928 B in 1s (4,065 B/s)
Selecting previously unselected package apt-transport-https.
(Reading database... 139578 files and directories currently installed.)
Preparing to unpack .../apt-transport-https_3.2.0_all.deb...
Unpacking apt-transport-https (3.2.0)...
```

Figure 6 – Terminal output of `'sudo apt-get install apt-transport-https'` completing successfully, installing version 3.2.0.

With the package installed, apt finished its housekeeping (confirming the kernel is current and that no services or containers need restarting), and the actual repository line was written into the new `elastic-9.x.list` file.

Command

```
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] \
https://artifacts.elastic.co/packages/9.x/apt stable main" | \
sudo tee /etc/apt/sources.list.d/elastic-9.x.list
```

Purpose: Adds Elastic's repository to apt's sources list so its packages become installable.


```
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@elasticsearch:/home/malik# echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/9.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elastic-9.x.list
deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/9.x/apt stable main
root@elasticsearch:/home/malik#
```

Figure 7 – Terminal confirming apt housekeeping completed and the Elastic 9.x repository line was written to `/etc/apt/sources.list.d/elastic-9.x.list`.

5. Part 3 — Installing Elasticsearch

With the repository now registered and trusted, the actual Elasticsearch package could be installed. Elastic's documentation offers both a “latest” and a “specific version” install path; the latest version available at the time of this deployment was Elasticsearch 9.4.2, installed with a single combined update-and-install command.

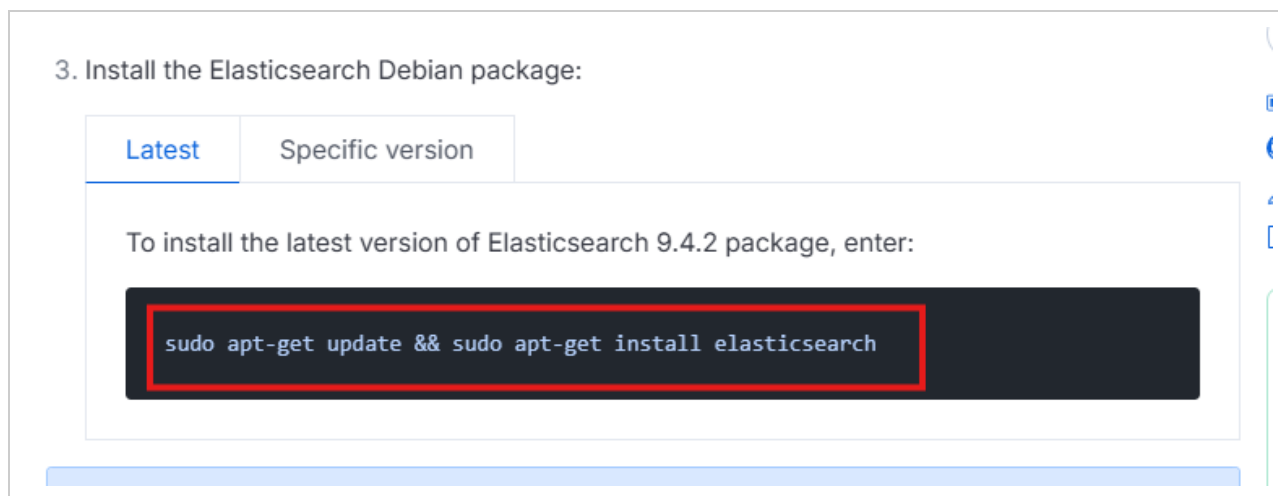


Figure 8 – Elastic's documentation showing the command to install the latest Elasticsearch 9.4.2 package.

Command — Refresh the package index and install Elasticsearch

```
sudo apt-get update && sudo apt-get install elasticsearch
```

Purpose: Refreshes the package list and installs Elasticsearch from the official repository.

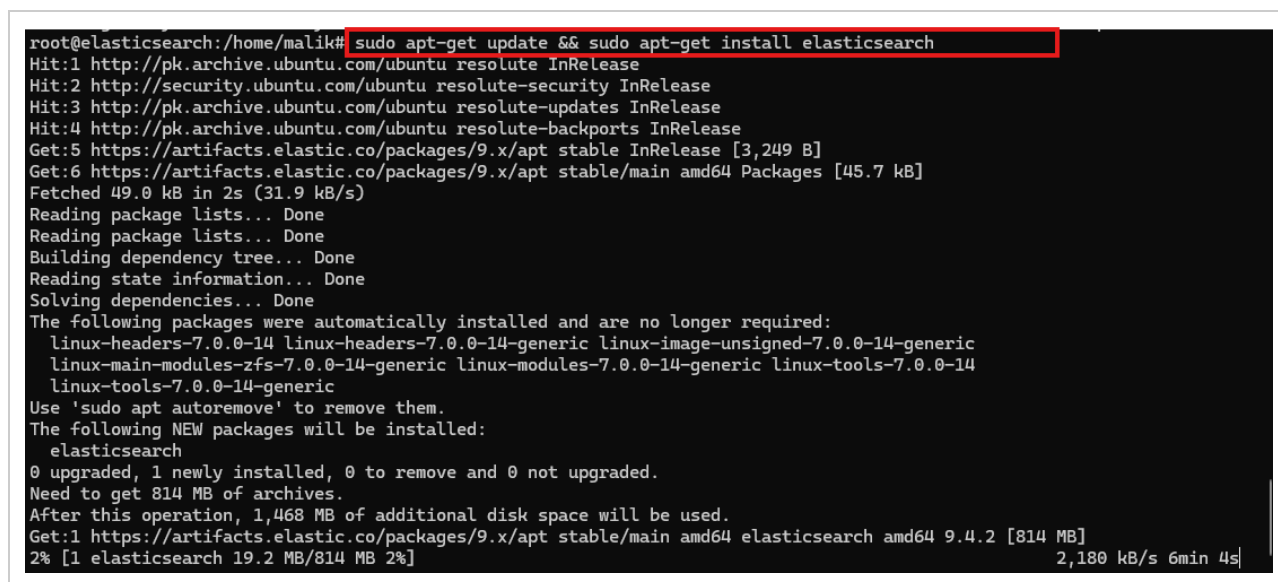


Figure 9 – Terminal showing the install command, apt resolving the new Elastic repository, and the 814 MB Elasticsearch package download beginning.

Step 5.1 – Security Auto-Configuration

Modern Elasticsearch installs perform security auto-configuration automatically as the package finishes setting up: TLS for the HTTP and transport layers is generated and enabled, and a random password is generated for the built-in elastic superuser. This password is printed once to the terminal at the very end of the install and is not displayed again afterward.

Important — save this output immediately

The elastic superuser password is shown only once, at the end of the Elasticsearch package installation. If it is lost, it has to be reset afterward with the `elasticsearch-reset-password` utility rather than simply looked up again.

```
Setting up elasticsearch (9.4.2)...
----- Security autoconfiguration information -----

Authentication and authorization are enabled.
TLS for the transport and HTTP layers is enabled and configured.

The generated password for the elastic built-in superuser is : dj7yZsTMh=oFGrCnCBrl

If this node should join an existing cluster, you can reconfigure this with
'/usr/share/elasticsearch/bin/elasticsearch-reconfigure-node --enrollment-token <token-here>'
after creating an enrollment token on your existing cluster.

You can complete the following actions at any time:

Reset the password of the elastic built-in superuser with
'/usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic'.

Generate an enrollment token for Kibana instances with
'/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana'.

Generate an enrollment token for Elasticsearch nodes with
'/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s node'.
```

Figure 10 – Security autoconfiguration output at the end of the Elasticsearch install: authentication and TLS are enabled, and the generated elastic superuser password is printed. The same output also lists the commands available for resetting the password and generating enrollment tokens for Kibana or additional Elasticsearch nodes.

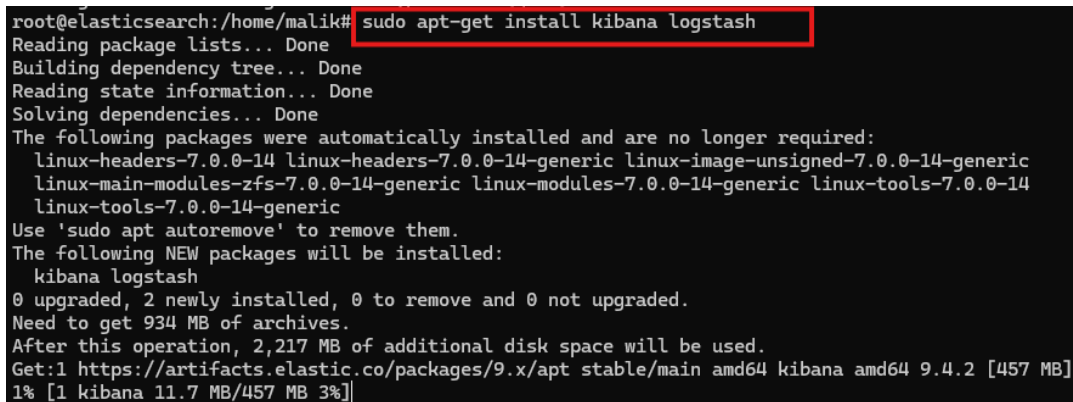
6. Part 4 — Installing Kibana & Logstash

Because the Elastic repository was already trusted and added to the system, installing the remaining two components — Kibana and Logstash — was a single apt-get command, with no further repository setup needed.

Command — *Install Kibana and Logstash*

```
sudo apt-get install kibana logstash
```

Purpose: Installs Kibana, the web dashboard, and Logstash, the log-processing pipeline.

A terminal window showing the execution of the command 'sudo apt-get install kibana logstash'. The output indicates that several linux-* packages are no longer required and will be removed. It then lists the new packages to be installed: kibana and logstash. It shows that 0 packages will be upgraded, 2 are newly installed, and 0 are not upgraded. The total size of the packages is 934 MB. The terminal output is as follows:

```
root@elasticsearch:/home/malik# sudo apt-get install kibana logstash
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14 linux-headers-7.0.0-14-generic linux-image-unsigned-7.0.0-14-generic
  linux-main-modules-zfs-7.0.0-14-generic linux-modules-7.0.0-14-generic linux-tools-7.0.0-14
  linux-tools-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  kibana logstash
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 934 MB of archives.
After this operation, 2,217 MB of additional disk space will be used.
Get:1 https://artifacts.elastic.co/packages/9.x/apt/stable/main amd64 kibana amd64 9.4.2 [457 MB]
1% [1 kibana 11.7 MB/457 MB 3%]
```

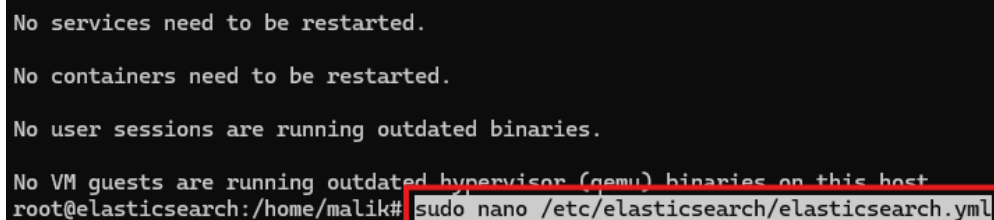
Figure 11 – Terminal showing ‘sudo apt-get install kibana logstash’ resolving both packages and beginning the 934 MB combined download.

Once the install finished and apt completed its usual housekeeping checks, the next step was to start editing the Elasticsearch configuration file directly.

Command — *Open the Elasticsearch configuration file*

```
sudo nano /etc/elasticsearch/elasticsearch.yml
```

Purpose: Opens Elasticsearch's main configuration file for editing.

A terminal window showing the output of apt housekeeping checks. It states that no services, containers, or user sessions need to be restarted, and that no VM guests are running outdated hypervisor (qemu) binaries. The terminal then shows the command 'sudo nano /etc/elasticsearch/elasticsearch.yml' being entered. The output is as follows:

```
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host
root@elasticsearch:/home/malik# sudo nano /etc/elasticsearch/elasticsearch.yml
```

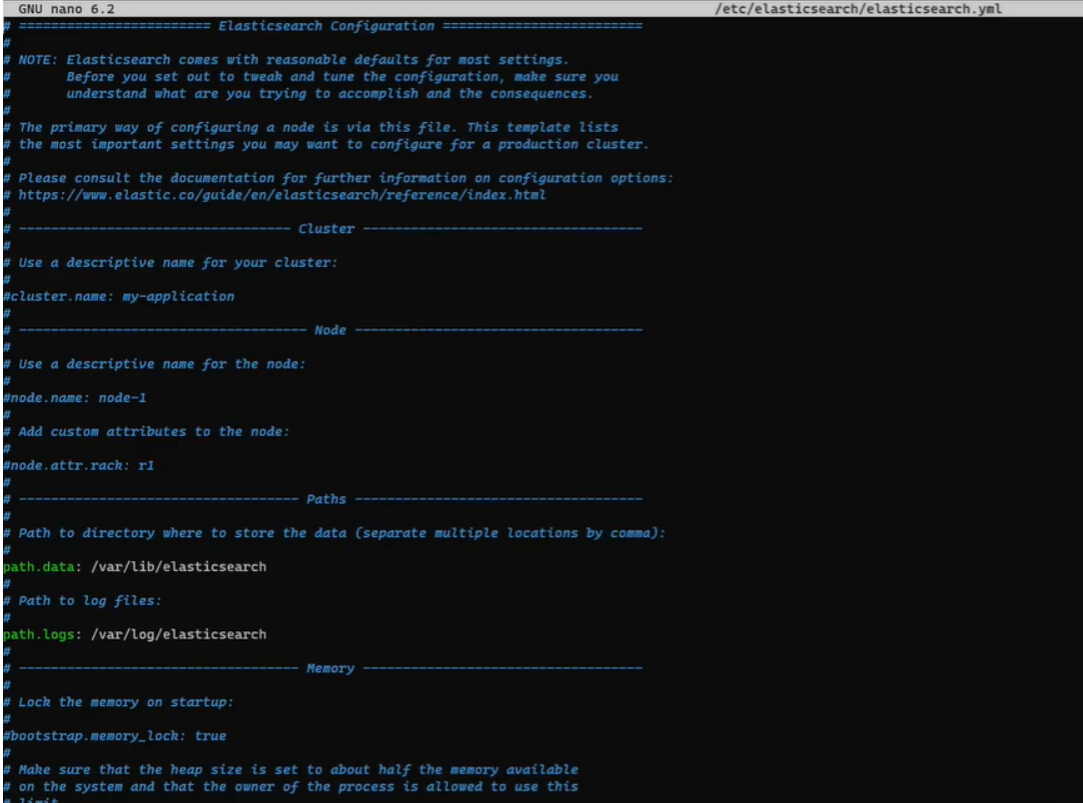
Figure 12 – Terminal confirming apt housekeeping and the command opening elasticsearch.yml in nano.

7. Part 5 — Configuring Elasticsearch (elasticsearch.yml)

By default, Elasticsearch ships with every meaningful setting commented out, relying on sane defaults and binding only to localhost. For this lab, three changes were made: giving the cluster a descriptive name, and exposing the node on the host's network interface so Kibana — and any browser on the same network — can reach it.

Step 7.1 – Reviewing the Default Configuration

The top of the file shows the default, fully-commented template: cluster.name, node.name, the data and log paths (already active and pointing at /var/lib/elasticsearch and /var/log/elasticsearch), and the memory-locking option.



```
GNU nano 6.2 /etc/elasticsearch/elasticsearch.yml
# ===== Elasticsearch Configuration =====
#
# NOTE: Elasticsearch comes with reasonable defaults for most settings.
#       Before you set out to tweak and tune the configuration, make sure you
#       understand what are you trying to accomplish and the consequences.
#
# The primary way of configuring a node is via this file. This template lists
# the most important settings you may want to configure for a production cluster.
#
# Please consult the documentation for further information on configuration options:
# https://www.elastic.co/guide/en/elasticsearch/reference/index.html
#
# ----- Cluster -----
# Use a descriptive name for your cluster:
#
#cluster.name: my-application
#
# ----- Node -----
# Use a descriptive name for the node:
#
#node.name: node-1
#
# Add custom attributes to the node:
#
#node.attr.rack: r1
#
# ----- Paths -----
# Path to directory where to store the data (separate multiple locations by comma):
#
#path.data: /var/lib/elasticsearch
#
# Path to log files:
#
#path.logs: /var/log/elasticsearch
#
# ----- Memory -----
# Lock the memory on startup:
#
#bootstrap.memory_lock: true
#
# Make sure that the heap size is set to about half the memory available
# on the system and that the owner of the process is allowed to use this
# limit
```

Figure 13 – elasticsearch.yml opened in nano, showing the default Cluster, Node, Paths, and Memory sections, all but the path settings still commented out.

Step 7.2 – Setting the Cluster Name

The cluster.name line was uncommented and given a custom value of offensivekernel in place of the default placeholder my-application. Every Elasticsearch node that wants to join the same cluster must share this exact name, so giving it a deliberate, descriptive value here matters more as soon as a second node is added later.

```

GNU nano 8.7.1 /etc/elasticsearch/elasticsearch.yml *
#
# ----- Cluster -----
#
# Use a descriptive name for your cluster:
#
cluster.name offensivekernel
#
# ----- Node -----
#
# Use a descriptive name for the node:
#
#node.name: node-1
#
# Add custom attributes to the node:
#
#node.attr.rack: r1
#
# ----- Paths -----
#
# Path to directory where to store the data (separate multiple locations by comma):
#
path.data: /var/lib/elasticsearch
#

```

Figure 14 – elasticsearch.yml with cluster.name uncommented and set to 'offensivekernel'.

Step 7.3 – Reviewing the Network Section

Scrolling further down the file reaches the Network block, which by default keeps Elasticsearch accessible only on localhost. The commented example shows network.host being set to a placeholder LAN address (192.168.0.1), with http.port left at its default value of 9200.

```

GNU nano 8.7.1 /etc/elasticsearch/elasticsearch.yml *
#
# Lock the memory on startup:
#
#bootstrap.memory_lock: true
#
# Make sure that the heap size is set to about half the memory available
# on the system and that the owner of the process is allowed to use this
# limit.
#
# Elasticsearch performs poorly when the system is swapping the memory.
#
# ----- Network -----
#
# By default Elasticsearch is only accessible on localhost. Set a different
# address here to expose this node on the network:
#
#network.host: 192.168.0.1
#
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
#
#http.port: 9200
#
# For more information, consult the network module documentation.
#
# ----- Discovery -----
#

```

Figure 15 – The Network section of elasticsearch.yml in its default, fully commented state.

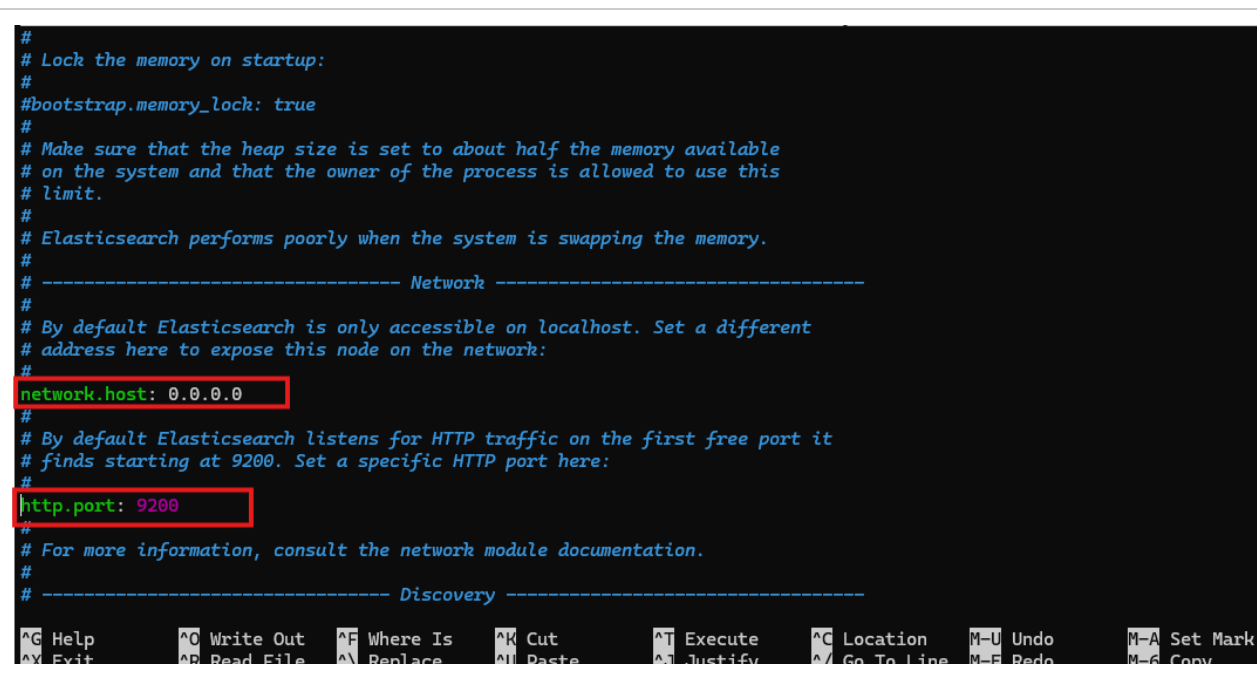
Step 7.4 – Exposing Elasticsearch on the Network

Both lines were uncommented and set explicitly: `network.host` was set to `0.0.0.0` so Elasticsearch listens on every network interface on the host rather than only on the loopback address, and `http.port` was set to `9200`, its standard REST API port. This is what allows Kibana — and any verification requests made later in a deployment like this — to reach Elasticsearch from outside the local machine.

Command — *Resulting configuration values*

```
network.host: 0.0.0.0
http.port: 9200
```

Purpose: Makes Elasticsearch reachable from other devices on the network, on port 9200.



```
#
# Lock the memory on startup:
#
#bootstrap.memory_lock: true
#
# Make sure that the heap size is set to about half the memory available
# on the system and that the owner of the process is allowed to use this
# limit.
#
# Elasticsearch performs poorly when the system is swapping the memory.
#
# ----- Network -----
#
# By default Elasticsearch is only accessible on localhost. Set a different
# address here to expose this node on the network:
#
network.host: 0.0.0.0
#
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
#
http.port: 9200
#
# For more information, consult the network module documentation.
#
# ----- Discovery -----
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark
^Y Exit      ^P Read File ^N Replace   ^U Paste     ^I Justify   ^_ Go To Line M-F Redo     M-G Copy
```

Figure 16 – `elasticsearch.yml` with `network.host` set to `0.0.0.0` and `http.port` set to `9200`, both now active.

8. Part 6 — Configuring Kibana (kibana.yml)

With Elasticsearch's network settings saved, nano was closed and reopened against Kibana's own configuration file. Kibana needs the equivalent network changes so its web interface can be reached from a browser instead of being restricted to the server itself.

Command — Open the Kibana configuration file

```
sudo nano /etc/kibana/kibana.yml
```

Purpose: Opens Kibana's configuration file so its network and connection settings can be edited.

```
No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@elasticsearch:/home/malik# sudo nano /etc/elasticsearch/elasticsearch.yml
root@elasticsearch:/home/malik# sudo nano /etc/kibana/kibana.yml
```

Figure 17 – Terminal confirming elasticsearch.yml was saved and exited, then opening /etc/kibana/kibana.yml in nano.

Step 8.1 – Reviewing the Default Kibana Settings

By default, Kibana's server.port is commented out at 5601 (its standard port), and server.host defaults to "localhost" — meaning, just like Elasticsearch, Kibana would normally refuse connections from any machine other than itself.

```
GNU nano 8.7.1 /etc/kibana/kibana.yml
# For more configuration options see the configuration guide for Kibana in
# https://www.elastic.co/guide/index.html

# ===== System: Kibana Server =====
# Kibana is served by a back end server. This setting specifies the port to use.
#server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
#server.host: "localhost"

# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the 'server.rewriteBasePath' setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""

# Specifies whether Kibana should rewrite requests that are prefixed with
# 'server.basePath' or require that they are rewritten by your reverse proxy.
# Defaults to 'false'.
#server.rewriteBasePath: false

# Specifies the public URL at which Kibana is available for end users. If
# 'server.basePath' is configured this URL should end with the same basePath.
#server.publicBaseUrl: ""

[ Read 183 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo     M-6 Copy
```

Figure 18 – kibana.yml in its default state: server.port commented at 5601, server.host commented at "localhost".

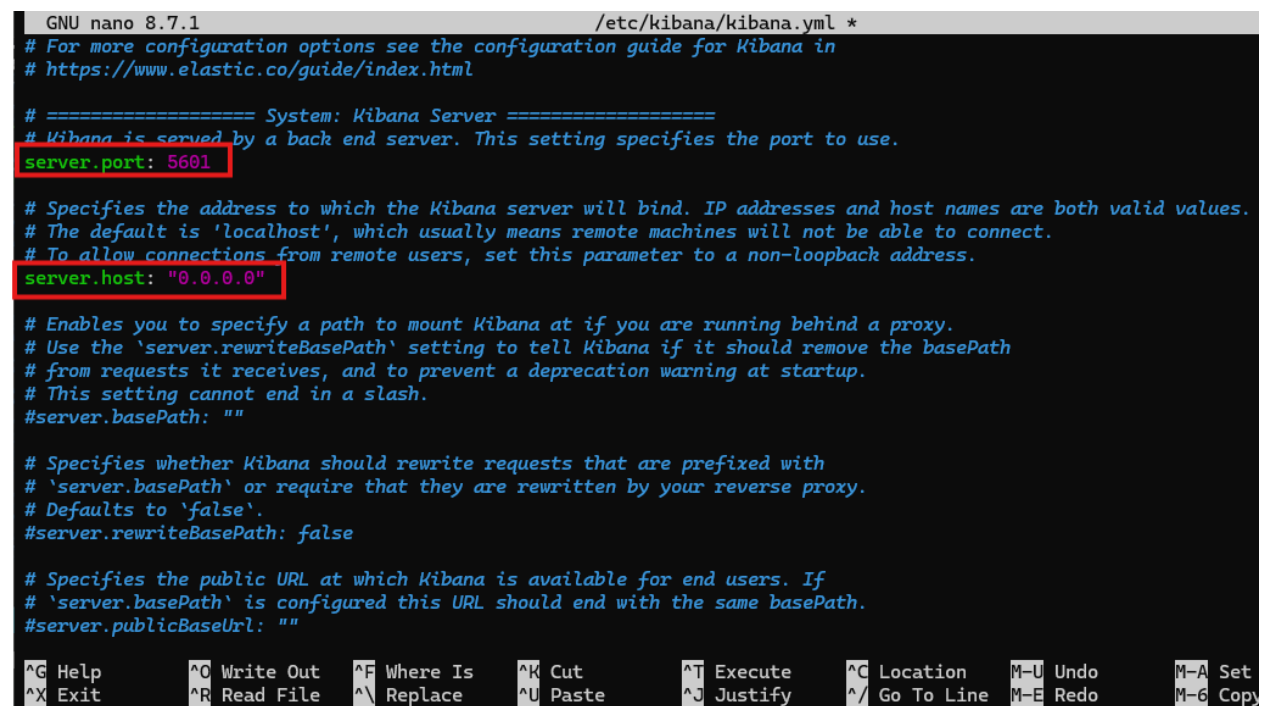
Step 8.2 – Exposing Kibana on the Network

Both settings were uncommented: `server.port` was confirmed at 5601, and `server.host` was changed from "localhost" to "0.0.0.0" so the Kibana web server accepts connections from any network interface, not just the loopback address.

Command — *Resulting configuration values*

```
server.port: 5601
server.host: "0.0.0.0"
```

Purpose: Makes the Kibana dashboard reachable from any device on the network, on port 5601.



```
GNU nano 8.7.1 /etc/kibana/kibana.yml *
# For more configuration options see the configuration guide for Kibana in
# https://www.elastic.co/guide/index.html

# ===== System: Kibana Server =====
# Kibana is served by a back end server. This setting specifies the port to use.
server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: "0.0.0.0"

# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the 'server.rewriteBasePath' setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""

# Specifies whether Kibana should rewrite requests that are prefixed with
# 'server.basePath' or require that they are rewritten by your reverse proxy.
# Defaults to 'false'.
#server.rewriteBasePath: false

# Specifies the public URL at which Kibana is available for end users. If
# 'server.basePath' is configured this URL should end with the same basePath.
#server.publicBaseUrl: ""

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line  M-E Redo      M-6 Copy
```

Figure 19 – kibana.yml with `server.port` active at 5601 and `server.host` changed to "0.0.0.0".

9. Part 7 — Starting and Verifying the Services

With both configuration files saved, the Elasticsearch and Kibana services were started through systemd and their status was checked to confirm each one came up cleanly before moving on to enrollment.

Command — *Start both services and check Elasticsearch's status*

```
systemctl start elasticsearch
systemctl start kibana
systemctl status elasticsearch
```

Purpose: Starts both services and confirms Elasticsearch is up and running.

```
No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@elasticsearch:/home/malik# sudo nano /etc/elasticsearch/elasticsearch.yml
root@elasticsearch:/home/malik# sudo nano /etc/kibana/kibana.yml
root@elasticsearch:/home/malik# systemctl start elasticsearch
root@elasticsearch:/home/malik# systemctl start kibana
root@elasticsearch:/home/malik# systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; disabled; preset: enabled)
   Active: active (running) since Sun 2026-06-28 08:49:34 UTC; 20s ago
  Invocation: 0c528cd7/cc448a8b79d14cb7121371e
     Docs: https://www.elastic.co
   Main PID: 18094 (server-launcher)
      Tasks: 103 (limit: 1827)
    Memory: 805.2M (peak: 1.1G, swap: 616.4M, swap peak: 616.4M)
       CPU: 1min 58.638s
    CGroup: /system.slice/elasticsearch.service
            └─18094 /usr/share/elasticsearch/lib/tools/server-launcher/server-launcher -p /var/run/elasticsearch/elast
              └─18144 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.n
                └─18167 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Jun 28 08:48:31 elasticsearch systemd[1]: Starting elasticsearch.service - Elasticsearch...
Jun 28 08:49:31 elasticsearch systemd-entrypoint[18094]: WARNING: A terminally deprecated method in sun.misc.Unsafe has
Jun 28 08:49:31 elasticsearch systemd-entrypoint[18094]: WARNING: sun.misc.Unsafe:arrayBaseOffset has been called by c
Jun 28 08:49:31 elasticsearch systemd-entrypoint[18094]: WARNING: Please consider reporting this to the maintainers of
Jun 28 08:49:31 elasticsearch systemd-entrypoint[18094]: WARNING: sun.misc.Unsafe:arrayBaseOffset will be removed in a
Jun 28 08:49:34 elasticsearch systemd[1]: Started elasticsearch.service - Elasticsearch.
lines 1-20/20 (END)
```

Figure 20 — ‘systemctl start elasticsearch / kibana’, followed by ‘systemctl status elasticsearch’ confirming the service is active (running), with its process tree and startup log.

Command — *Check Kibana's status*

```
systemctl status kibana
```

Purpose: Confirms Kibana is running and provides the setup URL needed to finish configuration.

```

root@elasticsearch:/home/malik# systemctl status kibana
● kibana.service - Kibana
   Loaded: loaded (/usr/lib/systemd/system/kibana.service; disabled; preset: enabled)
   Active: active (running) since Sun 2026-06-28 08:49:41 UTC; 1min 54s ago
     Invocation: b29198b06fe/4feeal27ad31662657a3
       Docs: https://www.elastic.co
    Main PID: 18246 (MainThread)
      Tasks: 11 (limit: 1827)
    Memory: 699.7M (peak: 699.7M, swap: 1.5M, swap peak: 1.5M)
       CPU: 37.416s
    CGroup: /system.slice/kibana.service
            └─18246 /usr/share/kibana/bin/./node/default/bin/node /usr/share/kibana/bin/./src/cli/kibana/dist

Jun 28 08:49:52 elasticsearch kibana[18246]: Native global console methods have been overridden in production environme
Jun 28 08:49:58 elasticsearch kibana[18246]: [2026-06-28T08:49:58.908+00:00][INFO ][root] Kibana is starting
Jun 28 08:49:59 elasticsearch kibana[18246]: [2026-06-28T08:49:59.008+00:00][INFO ][node] Kibana process configured wit
Jun 28 08:50:26 elasticsearch kibana[18246]: [2026-06-28T08:50:26.781+00:00][INFO ][plugins-service] The following plug
Jun 28 08:50:26 elasticsearch kibana[18246]: [2026-06-28T08:50:26.912+00:00][INFO ][http.server.Preboot] http server ru
Jun 28 08:50:27 elasticsearch kibana[18246]: [2026-06-28T08:50:27.160+00:00][INFO ][plugins-system.preboot] Setting up >
Jun 28 08:50:27 elasticsearch kibana[18246]: [2026-06-28T08:50:27.190+00:00][INFO ][preboot] "interactiveSetup" plugin >
Jun 28 08:50:27 elasticsearch kibana[18246]: [2026-06-28T08:50:27.248+00:00][INFO ][root] Holding setup until preboot s
Jun 28 08:50:37 elasticsearch kibana[18246]: i Kibana has not been configured.
Jun 28 08:50:37 elasticsearch kibana[18246]: Go to http://0.0.0.0:5601/?code=914966 to get started.

root@elasticsearch:/home/malik#

```

Figure 21 – ‘systemctl status kibana’ confirming the service is active (running), ending with the log line providing the setup URL and one-time code needed to finish configuring Kibana.

10. Part 8 — Enrolling Kibana with Elasticsearch

Kibana doesn't automatically trust an Elasticsearch cluster just because both are running on the same host — security auto-configuration means every connection has to be explicitly authorized. The setup URL printed in Kibana's logs opens a one-time onboarding wizard that asks for an enrollment token generated on the Elasticsearch side.

Step 10.1 – Opening the Kibana Setup Page

Navigating to the URL from Kibana's startup log — `http://192.168.44.134:5601/?code=914966` — in a browser loads the “Configure Elastic to get started” screen, prompting for an enrollment token pasted in from the terminal.

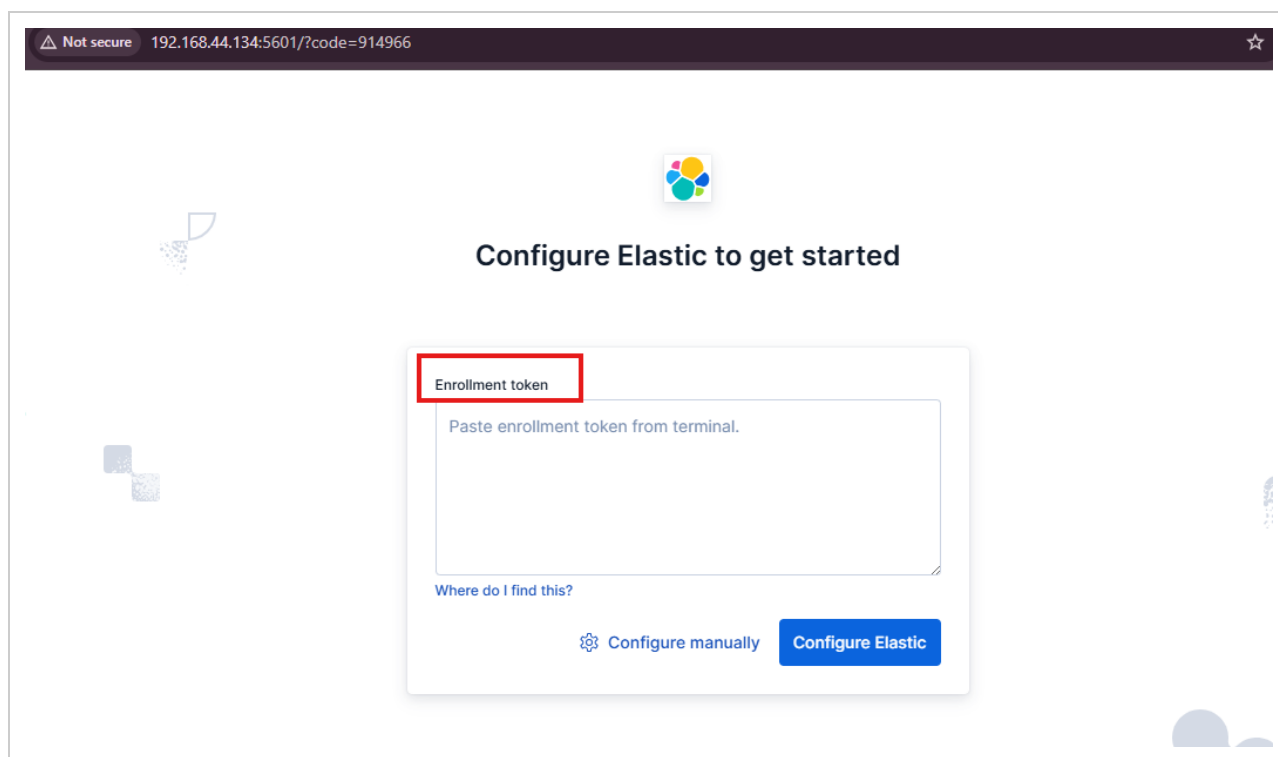


Figure 22 – The ‘Configure Elastic to get started’ screen in the browser, with an empty Enrollment token field waiting for input.

Step 10.2 – Generating the Enrollment Token

Back in the terminal, Elasticsearch's bundled enrollment-token utility was used to generate a token scoped specifically for a Kibana instance, exactly as suggested in the earlier security auto-configuration output (see Figure 10).

Command — *Generate an enrollment token for Kibana*

```
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
```

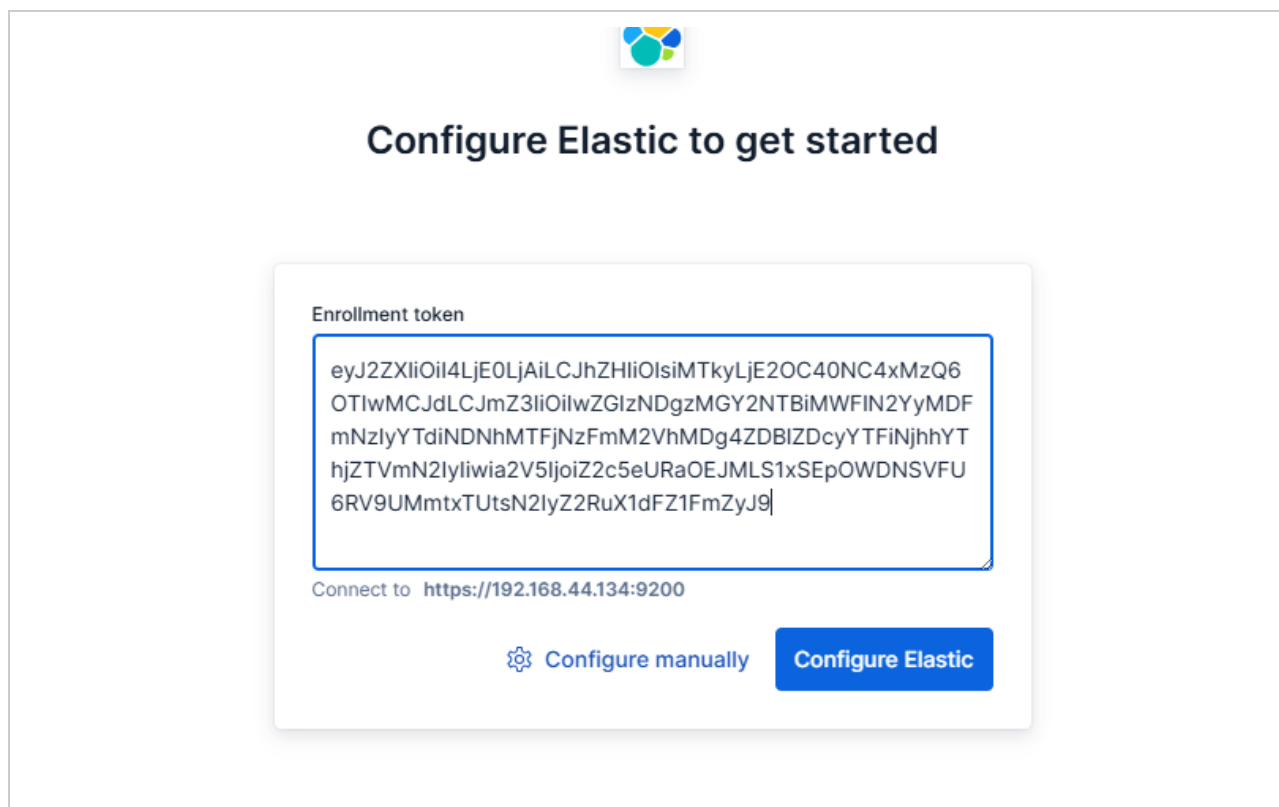
Purpose: Generates a one-time token Kibana uses to securely connect to Elasticsearch.

```
root@elasticsearch:/home/malik# sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
eyJ2ZXliOiI4LjE0LjAiLCJhZHIiOiIsImTKyLjE2OC40NC4xMzQ6OTIwMCJdLCJmZ3IiOiIwZGIzNDgzMGY2NTBiMWFjN2YyMDFmNzIyYTFiNDNhMTFjNzFmM2VhMDg4ZDBlZDcyYTFiNjhhYTJmN2IyIiwia2V5Ijo1Z2c5eURaOEJMLS1xSEpOWDNSVFU6RV9UMmtxTUtsN2IyZ2RuX1dFZ1FmZyJ9
root@elasticsearch:/home/malik#
```

Figure 23 – Terminal output of ‘elasticsearch-create-enrollment-token -s kibana’, producing the enrollment token string to be pasted into the browser.

Step 10.3 – Completing Enrollment in the Browser

The generated token was copied and pasted into the Enrollment token field in the browser. Kibana parses the token to discover the Elasticsearch endpoint to connect to — shown here as <https://192.168.44.134:9200> — before the Configure Elastic button is used to proceed.





Configure Elastic to get started

Enrollment token

eyJ2ZXliOiI4LjE0LjAiLCJhZHIiOiIsImTKyLjE2OC40NC4xMzQ6OTIwMCJdLCJmZ3IiOiIwZGIzNDgzMGY2NTBiMWFjN2YyMDFmNzIyYTFiNDNhMTFjNzFmM2VhMDg4ZDBlZDcyYTFiNjhhYTJmN2IyIiwia2V5Ijo1Z2c5eURaOEJMLS1xSEpOWDNSVFU6RV9UMmtxTUtsN2IyZ2RuX1dFZ1FmZyJ9

Connect to <https://192.168.44.134:9200>

 [Configure manually](#) [Configure Elastic](#)

Figure 24 – The enrollment token pasted into Kibana's setup screen, with Kibana confirming it will connect to <https://192.168.44.134:9200>.

After clicking through, Kibana runs its onboarding sequence automatically: saving the connection settings, starting its Elastic-side services, and finishing setup.

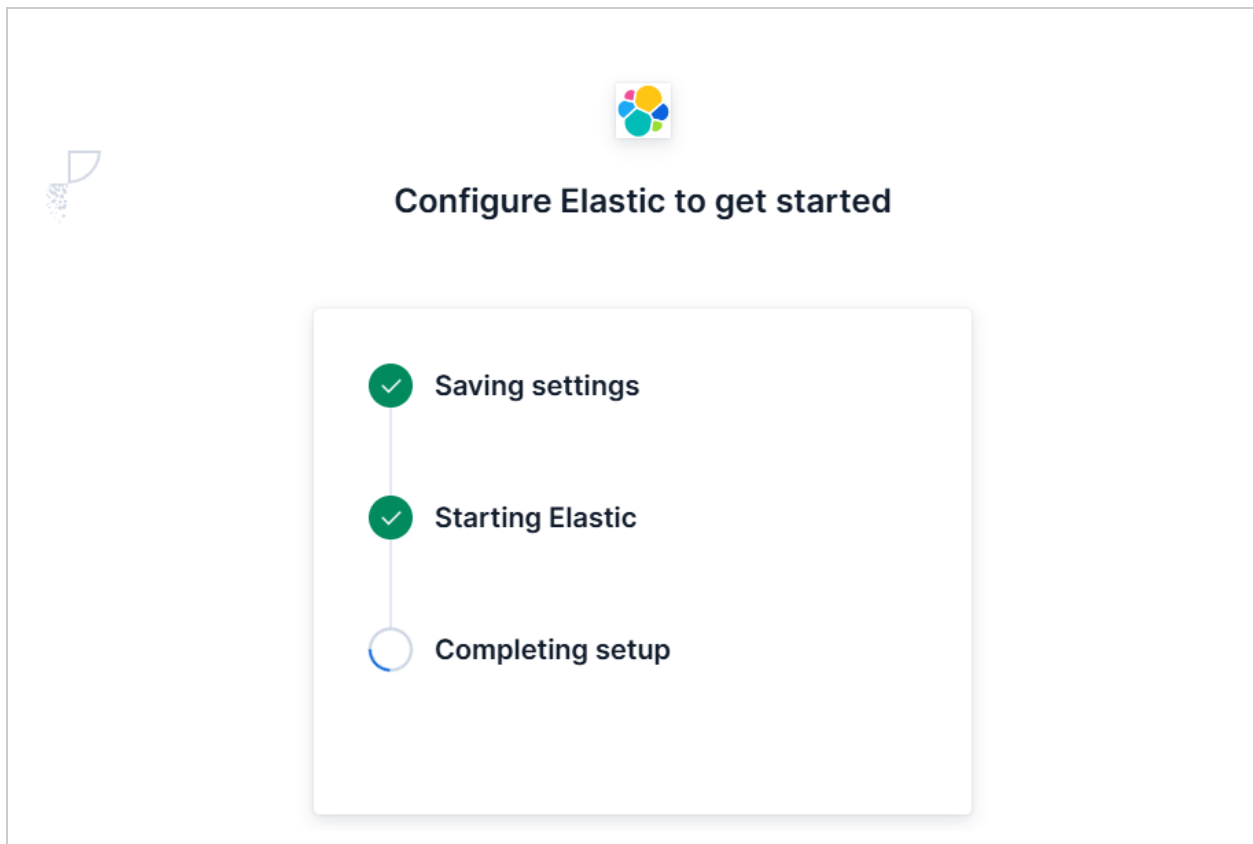


Figure 25 – Kibana's setup progress screen: 'Saving settings' and 'Starting Elastic' completed, with 'Completing setup' in progress.

11. Part 9 — First Login and Stack Verification

Once setup finishes, Kibana redirects to its standard login page. This is the same “Welcome to Elastic” screen that any user — analyst or administrator — will see every time they access the dashboard going forward.

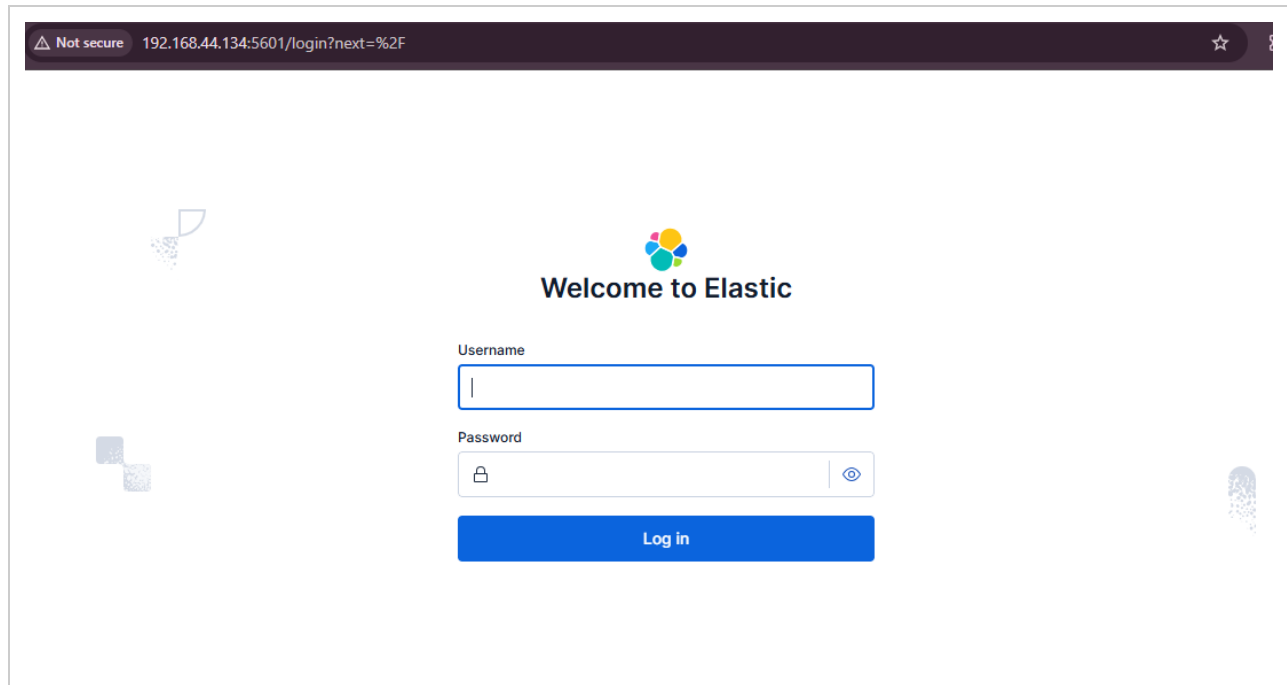


Figure 26 – The Kibana login page at /login, confirming the dashboard is now reachable over the network on port 5601.

The built-in elastic superuser account was used to log in for the first time, using the username elastic and the password that was generated and displayed during the Elasticsearch installation (Figure 10).



Welcome to Elastic

Username

Password

Log in

Figure 27 – The username 'elastic' entered in the login form.

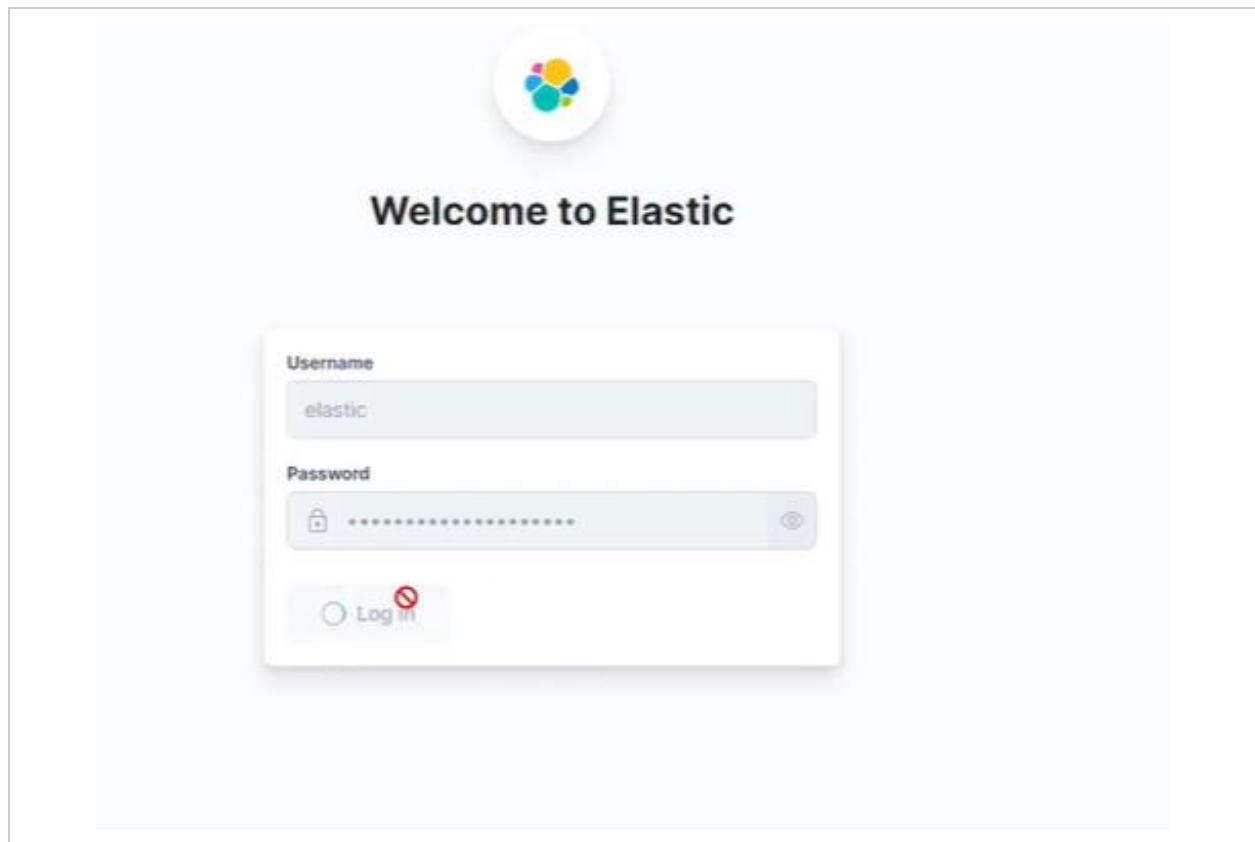


Figure 28 – The password field filled in (masked) and the Log in button being submitted.

Recommended hardening step

The elastic superuser is intended for initial setup and administration, not day-to-day use. In any deployment beyond a personal lab, the next steps would be to reset this password immediately after first login and create individually-named, role-scoped accounts for each analyst, using `elasticsearch-reset-password` and Kibana's built-in user and role management.

Step 11.1 – Confirming the Stack Is Fully Operational

A successful login lands on Kibana's Welcome home screen, which confirms that all three layers of the stack are correctly wired together: the browser can reach Kibana on port 5601, Kibana is authenticated against Elasticsearch on port 9200, and the cluster is healthy enough to serve the home page's solution tiles for Elasticsearch, Observability, Security, and Analytics.

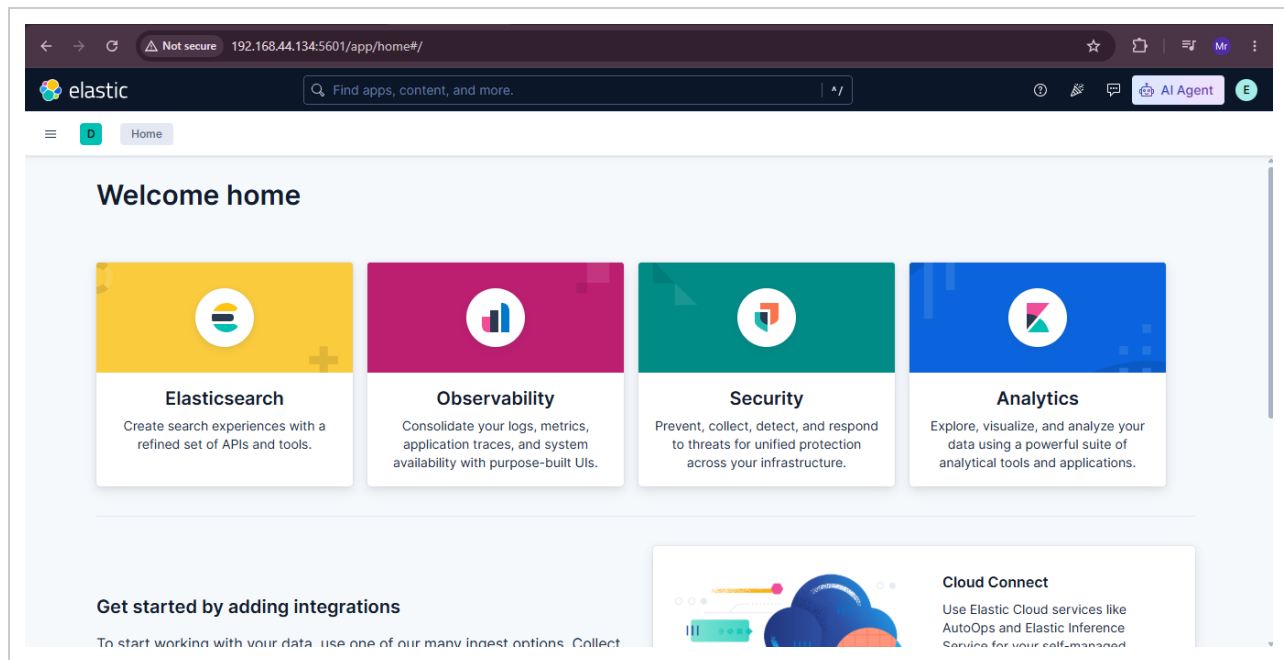


Figure 29 – The Kibana 'Welcome home' dashboard after a successful login, confirming Elasticsearch, Kibana, and the underlying cluster are fully operational.

12. Conclusion & Recommended Next Steps

This lab covered a complete, manual deployment of the Elastic Stack — Elasticsearch, Kibana, and Logstash — on a single Ubuntu Server host, following Elastic's official Debian/APT installation path from start to finish. Every stage was performed and verified independently: importing and trusting Elastic's signing key, registering the official package repository, installing each component, exposing both Elasticsearch and Kibana on the network, starting and confirming each service through systemd, and completing the secure enrollment handshake between Kibana and Elasticsearch through the browser-based setup wizard.

The result is a fully operational single-node Elastic Stack, with TLS and authentication enabled out of the box thanks to Elasticsearch's security auto-configuration, and a working Kibana dashboard reachable from anywhere on the local network.

Suggested Next Steps

- Reset the elastic superuser password and create named, role-based accounts for ongoing use, rather than continuing to use the default superuser day-to-day.
- Configure Logstash pipelines to ingest logs from real sources (syslog, Beats, application logs) and forward structured events into Elasticsearch for indexing.
- Install Elastic Agent or Beats on additional endpoints to begin centralizing logs from more than one host, moving from a single-node lab toward a small multi-host monitoring setup.
- Explore Kibana's Security and Observability solutions to build detection rules, alerts, and dashboards on top of the ingested data — turning this raw deployment into an actual working SIEM use case.
- Replace the self-signed TLS certificates generated during auto-configuration with CA-signed certificates before exposing the stack beyond an isolated lab network.

Documenting the installation end-to-end like this — including every configuration change and every verification step — builds the same operational habits expected in a real SOC environment, where standing up and maintaining the logging and search layer correctly is the foundation everything else (alerting, detection engineering, incident response) is built on.